

OPERA: Achieving Secure and High-performance OLAP with ParallElized HomomoRphic CompArisons

Qi HU[†], Wei CHEN[†], Tianxiang SHEN^{*}, Xin YAO, Gong ZHANG, Heming CUI^{*}, Siu-Ming YIU



The University of Hong Kong



HUAWEI

Huawei Technologies

Related Work - Encrypted OLAP system for Data Security



Encrypted OLAP systems can be categorized into two types → **Challenges: Security and Performance Limitations**

☞ **Traditional Cryptographic OLAP system**, e.g. CryptDB [SOSP'11], GaussDB [VLDB'21], MongoDB [Security'23]

✗ **Security**: Only encrypt sensitive data, keep PKs and filter columns unencrypted/deterministic (CPA attacks).

✓ **Performance**: High performance, which is close to plaintext performance.

☞ **Fully Homomorphic Encryption (FHE) OLAP system**, e.g. HEDA [VLDB'22], HE³DB [CCS'23]

✓ **Security**: Table is homomorphically encrypted, with encryption performed at client side.

✗ **Performance**: Poor performance (Homomorphic operations are slow)

	System	Security Level	SQL Functionality	Parallel Acceleration	Single Round Trip
Crypto-based system	MongoDB [Security'23]	Low	Full	✓	✓
	CryptDB [SOSP'11]	Low	Full	✗	✓
	ArX [VLDB'19]	Medium	Partial	✗	✗
Hybrid system	SIG-DB [PLOS'18]	Medium	Partial	✗	✓
FHE-based system	HEAP [ISCA'24]	High	Partial	✓	N/A
	HE ³ DB [CCS'23]	High	Full	✗	✓
	Opera [Our Work]	High	Full	✓	✓

Why do FHE-based OLAP system have extremely poor performance?

👉 A TPC-H Q6 query on 128 records takes 143 seconds to complete

Insight I: Multiple HomComps slow down performance

✗ HomComp is needed for each *record-predicate* pair

👉 E.g., N records with M predicates = $N \times M$ comparisons

✗ 90%+ of the query execution time

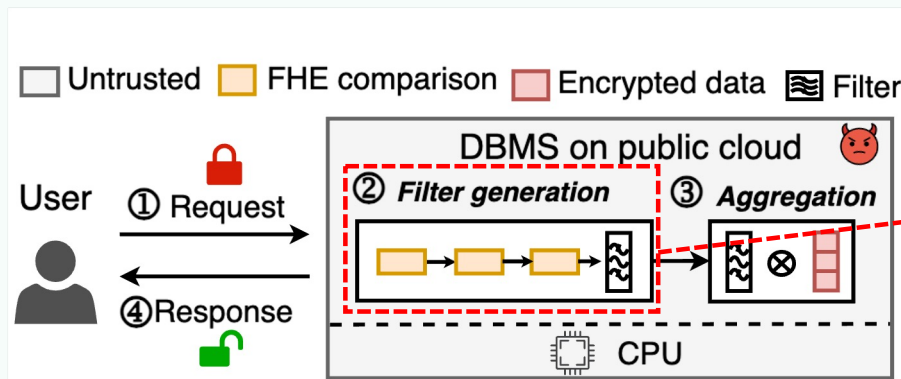


Fig 1: Query Process for FHE-based OLAP System

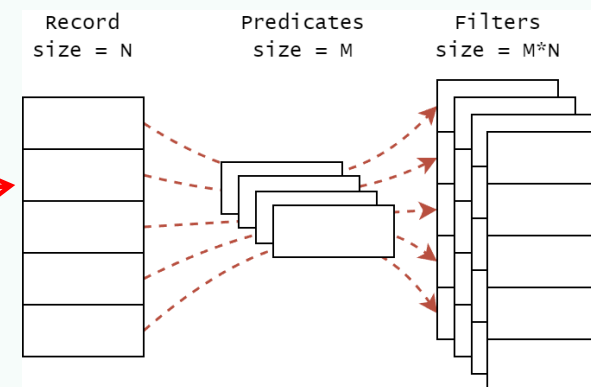


Fig 2: Filter is generated through HomComp operation on each record

Why do FHE-based OLAP system have extremely poor performance?

☞ A TPC-H Q6 query on 128 records takes 143 seconds to complete

Insight II: Only part of system can be parallelized

- ✓ Inter-parallelism: HomComps executed in parallel (*Layer 2*)
- ✓ Inner-parallelism: FHE operations can be paralleled (*Layer 0*)
- ✗ HomComp requires **sequential** FHE operations (*Layer 1*)

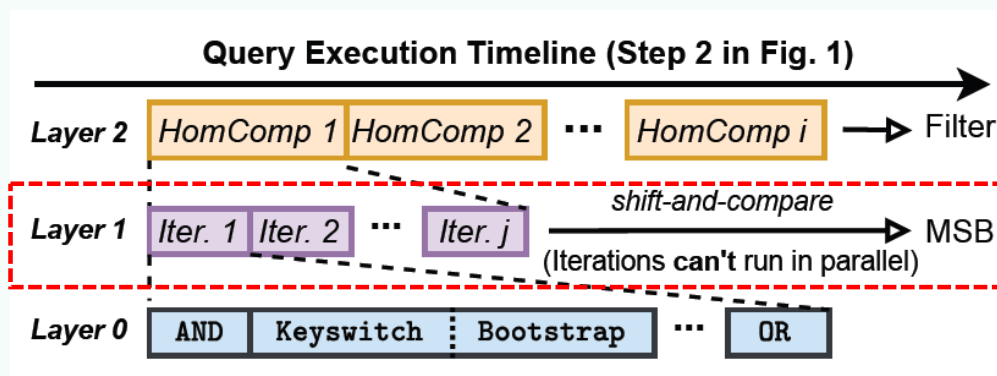


Fig 3: Layer decomposition for query execution

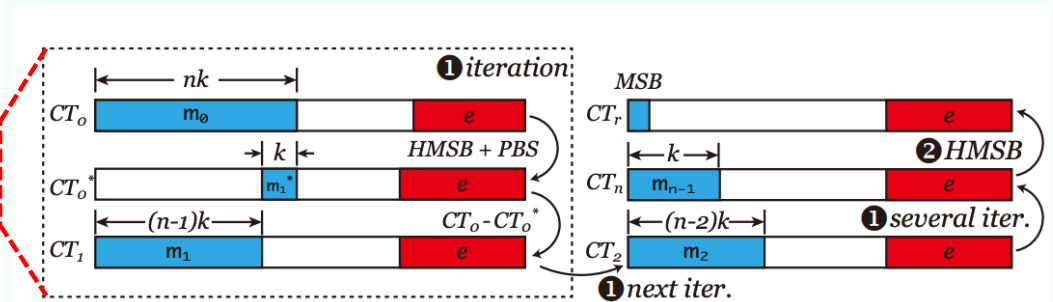


Fig 4: Traditional PBS-HMSB to extract the MSB of a ciphertext (*Layer 1*)

Contributions



1. We design and implement **Opera**, the first GPU-accelerated FHE-based OLAP system, with comprehensive bottleneck analysis and HomCache-driven Secure Comparison to enable full parallelization of homomorphic operations.
2. We propose a novel Density-driven Cache Eviction (DDE) Strategy that significantly outperforms traditional replacement strategies (e.g. LRU) for cached filter management.

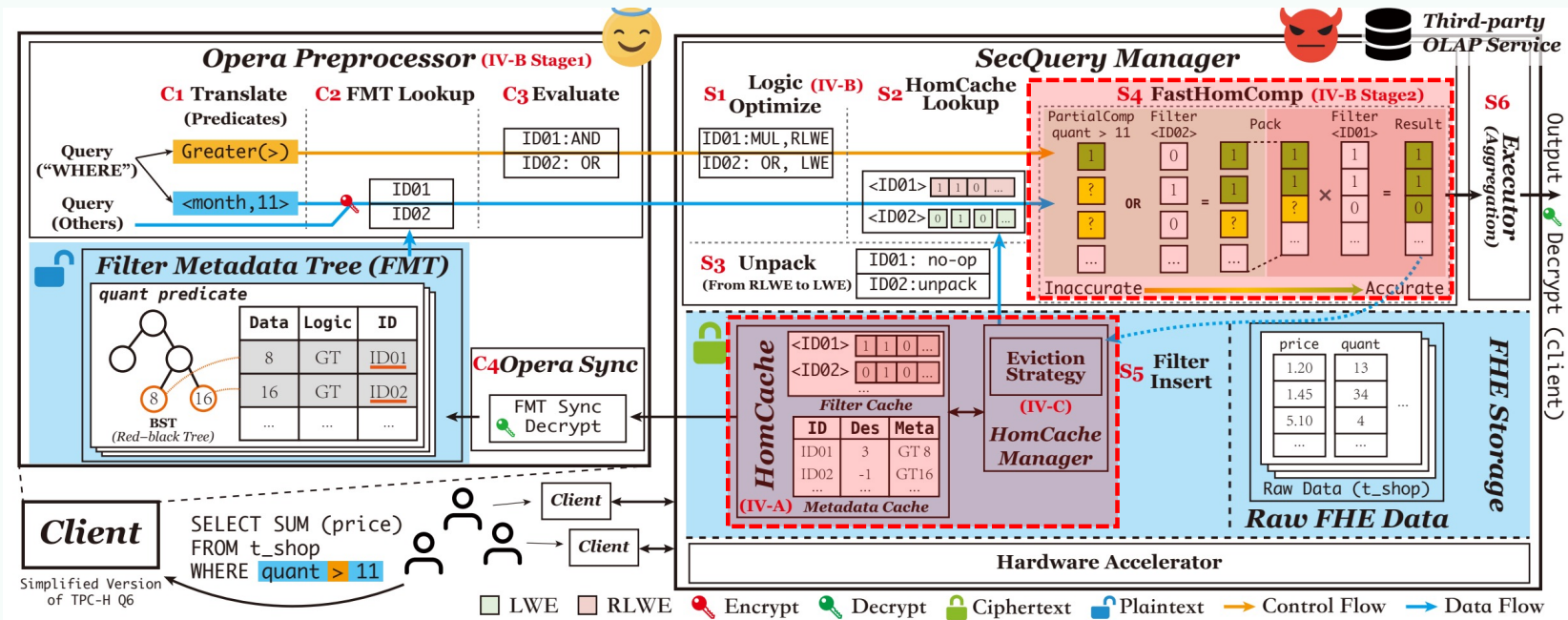


Fig 5: The architecture of **Opera**

Key Component I: HomCache-driven Secure Comparison



Fast Homomorphic Comparison (FHC)

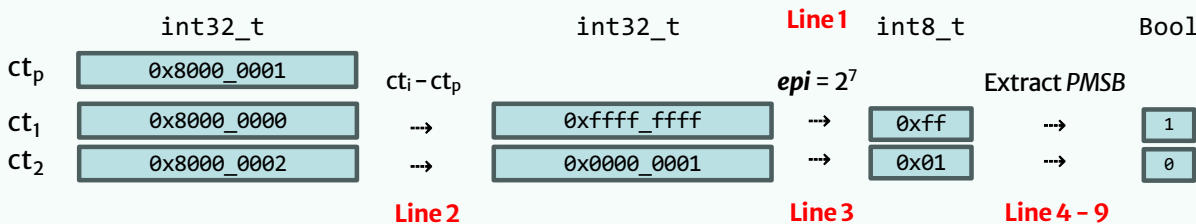
Insight: Sequential steps is determined by **data precision**

Solution:

✓ Use *low-precision data* to reduce **redundant iterations**

Partial Homomorphic Comparison (PHC) algorithm

E.g. Data stored as 32-bit, if we know 8-bit is sufficient, upper 24 bits can be ignored and only compare the lower bits.



Algorithm steps:

1. Determine the position of **PMSB** (Partial MSB), numerical distance (precision) is epi
2. Convert *high-precision data* into *low-precision format* ($low\text{-}precision\ data < 2^{epi}$)
3. Extract PMSB from *low-precision values*

position of PMSB (epi)

high-precision data into low-precision format

Extract PMSB

Algorithm 2: Partial Homomorphic Comparison (PHC)

Input: Comparison of *data* and *judge*

Input: Acceleration *filters*

Output: Comparison result in LWE format

// Calculate the position of PMSB (density in §4.1)

1 $space \leftarrow \lceil \log_2(filters[1] - filters[2]) \rceil$;

// Shift for discard high bits (obtain PMSB)

2 $pmsb \leftarrow HomSub(data, judge)$;

3 $pmsb \leftarrow HomShift(pmsb, Length(data) - space)$;

// Start to extract PMSB, similar to PBS-HMSB (§2.2)

4 **for** $space < space_{pmsb}$ **do**

// Iteration (Step 1 in Figure 3)

5 $ct_1 \leftarrow HomShift(pmsb, space - space_{pmsb})$;

6 $ct_2 \leftarrow HomSub(ct_1, HMSB(ct_1))$;

7 $ct_3 \leftarrow PBS(ct_2)$;

8 $pmsb \leftarrow HomSub(pmsb, ct_3)$;

// Update remaining space

9 $space \leftarrow space - space_{pmsb}$;

// Return the comparison result (Step 2 in Figure 3)

10 **return** $HMSB(pmsb)$;

Fig 6: Partial Homomorphic Comparison (PHC) algorithm

data precision

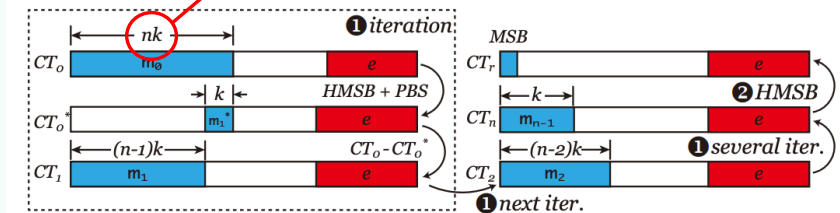


Fig 4: Traditional PBS-HMSB to extract the MSB of a ciphertext

Key Component I: *HomCache-driven Secure Comparison*



Fast Homomorphic Comparison (FHC)

🔍 Insight: Sequential steps is determined by **data precision**

Solution:

✓ Use *low-precision data* to reduce **redundant iterations**

🔧 Partial Homomorphic Comparison (PHC) algorithm

Challenge: Invalid assumptions $\rightarrow$ cannot control user *data precision*
 $\rightarrow$ induces **overflow risk** in HomComp

✓ Zone-based hierarchical correction (use *optimal (cached) filters*)

1. Partition into 3 regions using optimal filters (Fig. 8):

Gray Zone: PHC algorithm can calculate results accurately (without overflow)

Red Zone: May result in **false positives**, should be excluded

Green Zone: May result in **false negatives**, should be included

2. Apply correction using homomorphic logical processing

Red Zone: Exclude false positives $\rightarrow$ **AND** operation using optimal filter

Green Zone: Include false negatives $\rightarrow$ **OR** operation using optimal filter

Overflow!

Data	PHC Algorithm (11 - Data)	Filter1 (> 8)	Filter2 (> 16)	Result (> 11)
4	7 = 000 1 11 ₂	0	0	0
9	2 = 000 0 10 ₂	1	0	0
13	-2 = 111 1 0 ₂	1	0	1
34	-13 = 1 01 0 01 ₂	1	1	1

Fig 8: An example of comparison results of FHC algorithm with optimal filters. The **PMSB** result (3rd bit) from PHC algorithm is marked in blue, while red and green numbers indicate filter-based corrections.

AND Filter 1

OR Filter 2

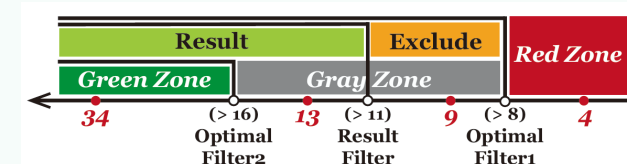


Fig 7: Three comparison zones in the FHE algorithm represented on a *Number Line*

Key Component II: *Density-driven Cache Eviction Strategy*



Density-driven Cache Eviction (DDE) Strategy

Challenge: Cache (optimal filter) size **grows linearly**

→ Eviction strategy

☞ Insight: Traditional strategy (e.g. LRU) results in performance deterioration

→ Not consider numerical distance (*density*)

→ *Density* affects performance (FHC algo.)

Solution:

✓ Model performance gains for each cached filter

Frequency of direct hit (accurate rate a)

Frequency of used in the correction step (usage rate u)

Density of the filter (density d)

✓ Formalize it as a **convex optimization problem**

$R(i)$: Benefit **decreases** if remove filter i

$I(i, j)$: Benefit **increases** if insert filter j following i -th

☞ k will be evicted if:

$$k = \operatorname{argmin}_{m \in \text{HomCache}} R(m) \wedge R(k) < I(i, j)$$

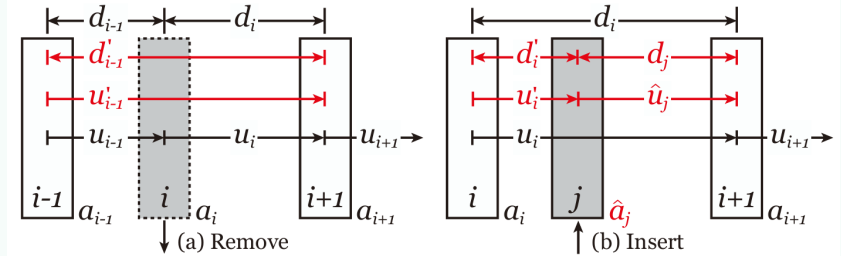


Fig 9: An example of the density model for the filters is in the HomCache: (a) filter i removal, (b) filter j insertion after i .

$$B_{corr}^{before}(i) = u_{i-1}(D_{ori} - D_{i-1}) + u_i(D_{ori} - D_i) \quad (2)$$

$$B_{corr}^{after}(i) = (u_{i-1} + u_i)(D_{ori} - D(d_{i-1} + d_i)) \quad (3)$$

$$R(i) = B_{corr}^{before}(i) - B_{corr}^{after}(i) + B_{hit}(i) \quad (\text{Remove})$$

Fig 10: The decrease of the benefit after removing the filter i .

$$\hat{a}_j = \frac{1}{2}(a_i + a_{i+1}) \quad (4)$$

$$\hat{u}_j = \frac{d_j}{d_i + d_{i+1}} \left(\frac{u_{i+1}}{d_{i+1}}(d_i - d_j) + \frac{u_i}{d_i}(d_{i+1} + d_j) \right) \quad (5)$$

$$\hat{B}_{hit}(j) = \hat{a}_j(D_{ori} - MULT) \quad (6)$$

$$B_{corr}^{before}(i) = u_i(D_{ori} - D_i) \quad (7)$$

$$\hat{B}_{corr}^{after}(i, j) = (u_i - \hat{u}_j)(D_{ori} - D(d_i - d_j)) + \hat{u}_j(D_{ori} - D_j) \quad (8)$$

$$I(i, j) = \hat{B}_{corr}^{after}(i, j) - B_{corr}^{before}(j) + \hat{B}_{hit}(j) \quad (\text{Insert})$$

Fig 11: The benefit increases after inserting the filter j following the i -th.

Implementation and Evaluation Setup



Implementation

- We developed our system based on the HE³DB codebase and implemented it using GPU-accelerated FHE libraries, including both *TFHE* and *CKKS* schemes.
- *Phantom* [TDSC'24] library for CKKS GPU implementation.
- Created a CUDA version of TFHEpp for TFHE.
- We implemented several optimizations for the GPU-accelerated FHE libraries, including *kernel fusion* and *multi-dimensional parallel* execution.

Evaluation Setup

- **Hardware.** The system utilizes dual Intel Xeon Silver 4216 CPUs (2.10GHz, 32 cores, 64 threads), 128GB of RAM, and an **NVIDIA A100 GPU**.
- **Settings.** *Opera* employs identical FHE parameters as the baseline system, maintaining a security level of at least 128 bits ($\lambda = 128$), with LWE dimension of 672 ($n = 672$) and RLWE dimension of 65536 ($N = 65536$).
- **Workload.** We evaluated our system using the TPC-H, with the *lineitem* table configured to a default record size of 2^{10} and *Opera*'s cache size set to 1.2GB.

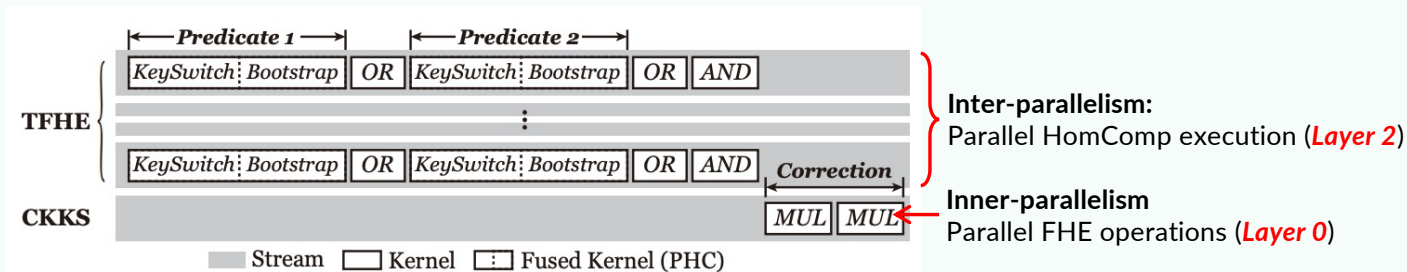


Fig 12: The example of the GPU acceleration process.

End-To-End Evaluations



End-To-End Performance Analysis

- **HEDA & HE³DB**: baseline OLAP system (in CPU)
 - **Opera-NC**: only leverages GPU for acceleration
 - **Opera-base**: enhances with cache optimization only (*FHC algo.*)
 - **Opera**: further optimization for *Opera-base*
- Overall performance improvement: approximately **200-9000x**
 - ✎ GPU acceleration brings about **110x** improvement (**HE³DB** vs **Opera-NC**)
 - ✎ Cache-driven homomorphic comparison yields **2-24x** performance improvement (**Opera-NC** vs **Opera**)
- Ablation Study of **Opera**'s Components (with Q6 Example)
 - ✎ For **HomComp (P1)**: about **6x** improvement (**Opera-NC** vs **Opera**)
 - ✎ For **LWE-Correct (P1.2)**: **50%** of total **HomComp** latency (**Opera-base**)
 - ✎ Use **RLWE-Correct (P1.3)**: **30%** overall latency reduction (**Opera**)
 - ✎ Optimization: Convert **LWE-Correct** to **RLWE-Correct (S1)**

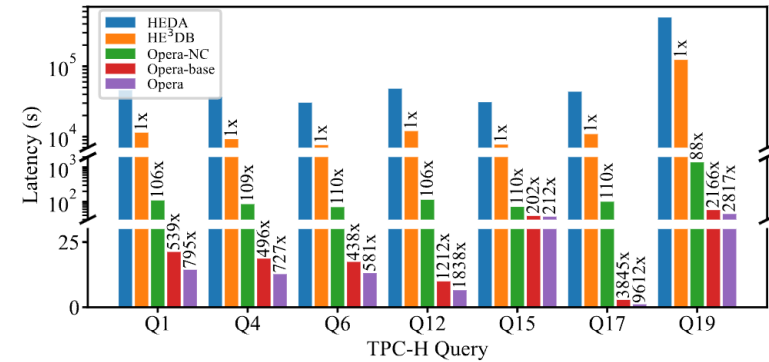


Fig 14: End-to-end Performance of TPC-H.

Record size	HE ³ DB			OPERA-NC		
	2 ⁸	2 ¹⁰	2 ¹²	2 ⁸	2 ¹⁰	2 ¹²
P1: HomComp	487.4	1865.9	7449.9	5.1	16.8	68.3
P2: Packing	73.1	153.0	152.5	0.3	1.1	1.1
P3: Aggregation	1.1	1.1	1.4	<0.1	<0.1	<0.1
End-to-end (P1~P3)	561.6	2020.0	7603.8	5.4	17.9	69.4

Record size	OPERA-base			OPERA		
	2 ⁸	2 ¹⁰	2 ¹²	2 ⁸	2 ¹⁰	2 ¹²
P1: HomComp (FHC)	1.3	4.1	16.3	0.9	3.1	12.0
P1.1: PHC (§4.2)	0.6	2.0	7.8	0.6	2.0	7.8
P1.2: LWE-Correct	0.7	2.1	8.5	0.3	1.0	4.2
P1.3: RLWE-Correct	N/A	N/A	N/A	<0.1	<0.1	<0.1
P2: Packing	0.4	1.1	1.1	0.4	1.1	1.1
P3: Aggregation	<0.1	<0.1	<0.1	<0.1	<0.1	<0.1
End-to-end (P1~P3)	1.7	5.2	17.4	1.3	4.1	13.1

Fig 15: End-to-end latency breakdown of **Opera (Q6)**.

End-To-End Evaluations



Performance with different cache eviction strategies

- Overall performance improvement
- DDE achieves about **14%** latency improvement over LRU strategy
- Latency tends to stabilize when cache size reaches $\sim 1.2\text{GB}$ (Fig. 18)
- Ablation study when **HomCache** Preheating and Stability
- **300** requests for *uniform* and **150** for *Zipfian* preheating
- DDE moving average latency stabilizes in **15-20s** after preheating
- But LRU moving average latency often **exceeds 20s**
- DDE has **fewer evictions** after preheating (Fig. 19)
 - Reduced the FMT synchronization overhead (**C4**)

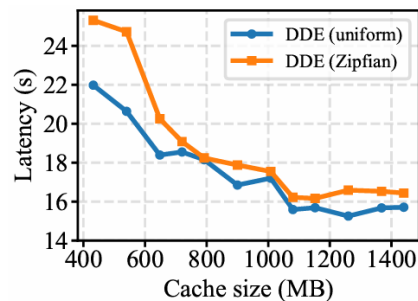


Fig 18: Performance of FHC with different cache sizes.

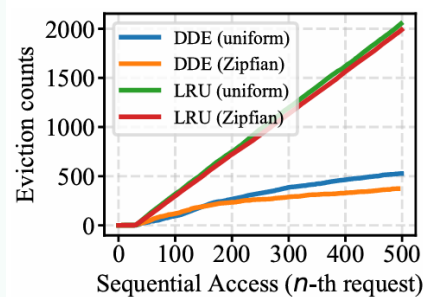


Fig 19: Number of evictions with different eviction strategies.

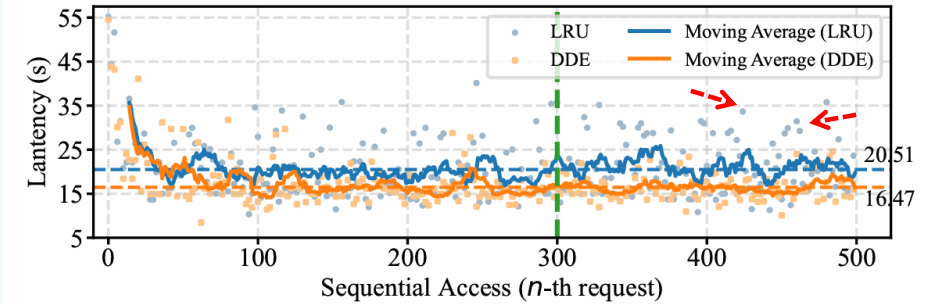


Fig 16: The latency of *HomComp* (FHC) with *uniform* distribution in different cache eviction strategies.

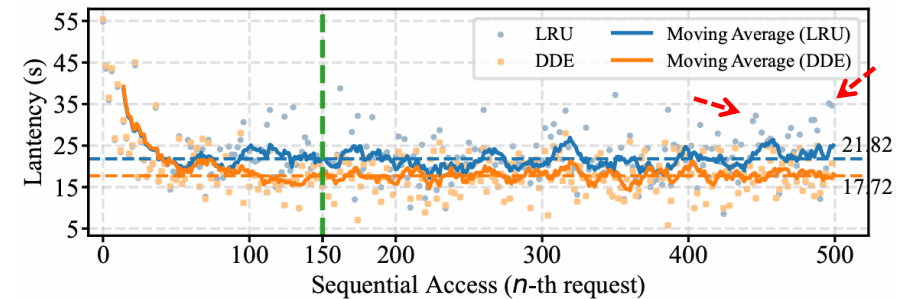


Fig 17: The latency of *HomComp* (FHC) with *Zipfian* distribution in different cache eviction strategies.

Conclusion



Opera presents the first FHE-based OLAP system that combines database caching with GPU acceleration. By integrating **HomCache** and **Fast Homomorphic Comparison (FHC)** algorithm, **Opera** effectively addresses parallelization challenges in homomorphic operations, achieving secure and efficient data processing.

Our Key contributions:

- **Opera** transforms sequential homomorphic comparisons into parallel operations via *HomCache* and *FHC algorithm*, **enabling efficient GPU acceleration**.
- **Opera** uses density-driven cache management to optimize comparison result reuse, achieving **stable and efficient acceleration under limited cache capacity**.
- Evaluations demonstrate **Opera**'s performance with up to 9000× speedup in query processing compared to existing systems, while maintaining stable performance across various workloads.

**Thanks for listening
and questions are welcome!**

Speaker: Qi HU